Performance Characterization of Recursive Quicksort on Out-of-Order Architectures

Course: EEL 6764 - Computer Architecture
Institution: USF Bellini College of Artificial Intelligence, Cybersecurity and Computing
Team: Srikanth Akkaru, Mandar Dhamale, Johnathan Gutierrez-diaz, Sri Satya Sudarshan Varma Indukuri

1. Executive Summary & Hypothesis

Recursive Quicksort is a canonical divide-and-conquer algorithm with O(n log n) average-case time complexity. While elegant in software, it produces irregular execution and memory access patterns in hardware. Specifically, Quicksort alternates between sequential array scans during the `partition()` phase and non-contiguous memory bounds adjustments during the recursive tree descent. Furthermore, the `if (arr[j] < pivot)` instruction creates dense, data-dependent conditional branches leading to frequent branch divergence.

Hypotheses:

- **Control Flow:** Branch mispredictions will be frequent due to random pivot selections and unpredictable data-dependent comparisons, creating pipeline flushes.
- **Memory Locality:** Cache performance will suffer due to mixed locality—while the partition phase exhibits sequential locality, the recursive jumps reduce temporal locality across recursive subproblems and introduce irregular access patterns.
- **Scaling Constraints:** Within the evaluated cache size range, performance will show minimal sensitivity to capacity, suggesting limited benefit from increasing L2 size for this workload.

2. Experimental Methodology

To ensure accurate, single-variable evaluations, simulations were executed using the **gem5 architectural simulator** (Syscall Emulation Mode). We standardized on uniformly random arrays of **n = 100,000 integers**. This dataset (~400kB footprint) was explicitly chosen to purposefully exceed the baseline L2 cache size (256kB), ensuring realistic capacity pressure.

To filter out initialization noise, we injected x86 inline assembly (`m5_dump_reset_stats`) directly into the C benchmark. This created a strict Region-of-Interest that captured metrics only during the sorting execution.

Limitations & Future Work: This evaluation is limited to uniformly random input distributions. Future work should evaluate the microarchitectural behavior of sorted,

reverse-sorted, or heavily skewed arrays, which alter the predictability of branches and the recursion depth.

3. Metric Definitions & Clarifications

To formally evaluate the architectures, we utilized the following standardized metrics:

- **IPC (Instructions Per Cycle):** The average number of instructions retired per clock cycle. Higher is better.
- **CPI (Cycles Per Instruction):** The average number of clock cycles required to execute one instruction. For single-core systems, CPI is the inverse of IPC (CPI ≈ 1 / IPC). Both are reported to provide intuition on execution efficiency versus stall latency.
- **Branch MPKI (Mispredictions Per Kilo-Instruction):** The number of branch mispredictions per 1,000 committed instructions. Normalizing by instruction count standardizes comparisons across different runs.
- **L2 Demand MPKI (Misses Per Kilo-Instruction):** The number of L2 demand cache misses per 1,000 committed instructions.
- **L2 Miss Rate (%):** The ratio of L2 demand misses to total L2 demand accesses.

4. Configuration & Isolation Strategy

To provide causal attribution, we established a Baseline and varied exactly **one parameter at a time**.

- **Baseline:** `O3CPU` (x86, 3GHz), `TournamentBP`, 256kB L2 Cache, Standard Memory
- **Config 1 (Control Flow):** Same as Baseline, but swapped branch predictor to `LTAGE`
- **Config 2 (Compute Pipeline):** Same as Baseline, but swapped processor to `TimingSimple` (In-Order)
- **Config 3 (LLC Capacity):** Same as Baseline, but doubled L2 cache to **512kB**
- **Config 4 (Spatial Prefetching):** Same as Baseline, but attached a **StridePrefetcher** to the L2 Cache

5. Performance Results & Metric Breakdown

Configuration	IPC	CPI	Exec Ticks (M)	Branch MPKI	L2 Demand MPKI	L2 Miss %
Baseline (O3, 256k)	0.8613	1.161	2747.0	1.000	0.205	20.04%
Config 1 (LTAGE)	0.8720	1.147	2713.2	0.105	0.203	19.88%
Config 2 (In-Order)	0.5933	1.685	3987.7	N/A	N/A	18.63%
Config 3 (512k L2)	0.8616	1.161	2746.0	1.000	0.200	19.65%
Config 4 (Prefetch)	0.8613	1.161	2747.0	1.000	0.205	20.04%

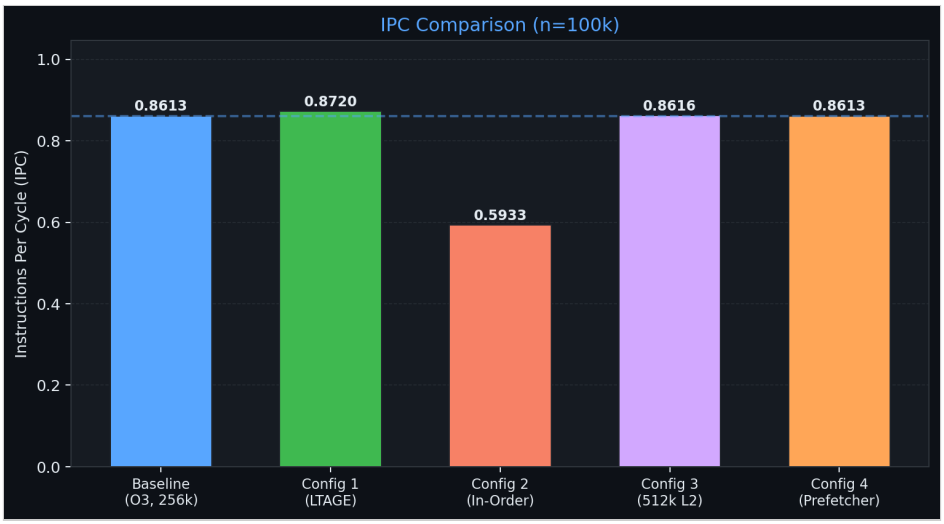

Figure 1: Baseline IPC against evaluated architectural modifications. The graph illustrates that switching to an In-Order core causes a severe IPC drop, while other configurations yield minimal IPC variance.

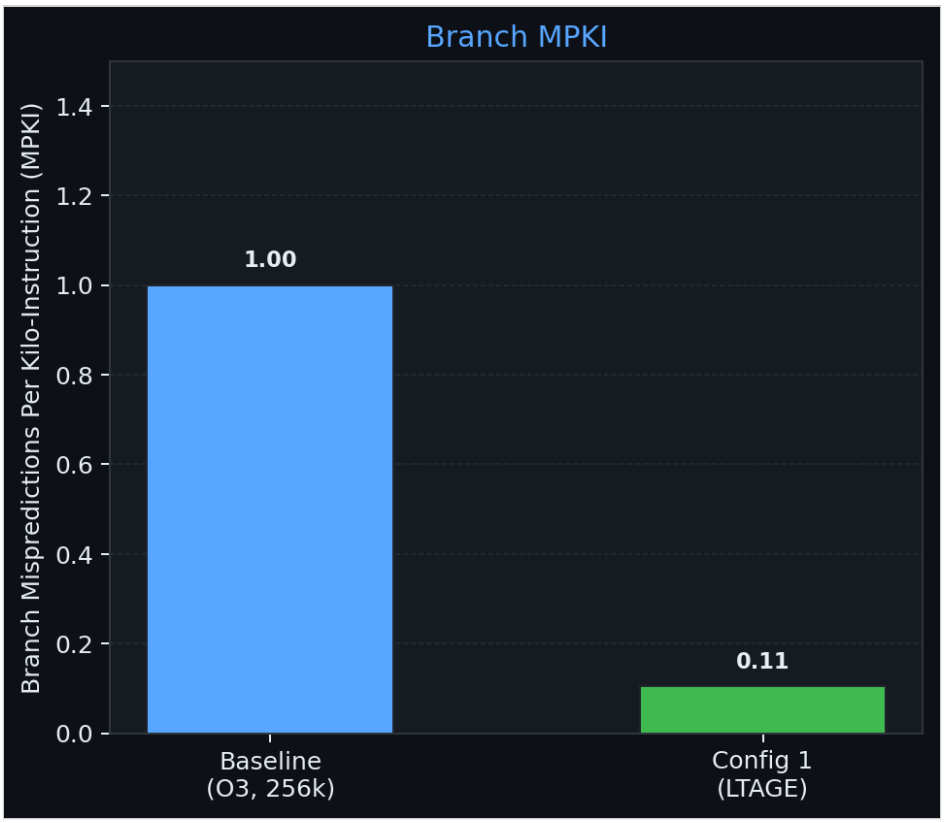

Figure 2: Branch Mispredictions Per Kilo-Instruction. The graph shows a significant reduction in Branch MPKI when using LTAGE, confirming improved prediction accuracy. However, the modest IPC gain suggests branch misprediction is not the dominant bottleneck.

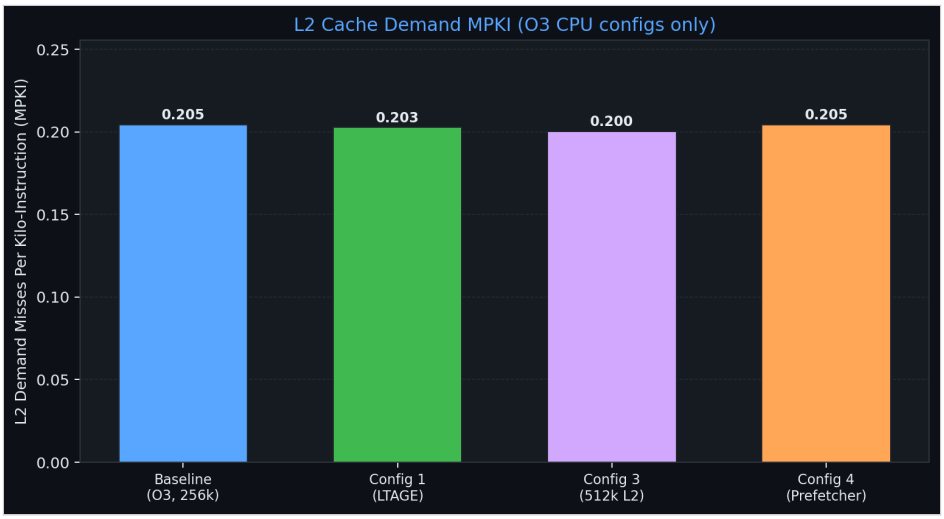

Figure 3: L2 Demand MPKI (O3 configurations only). The graph demonstrates that doubling L2 capacity or adding a stride prefetcher has a nearly imperceptible impact on L2 misses per instruction.

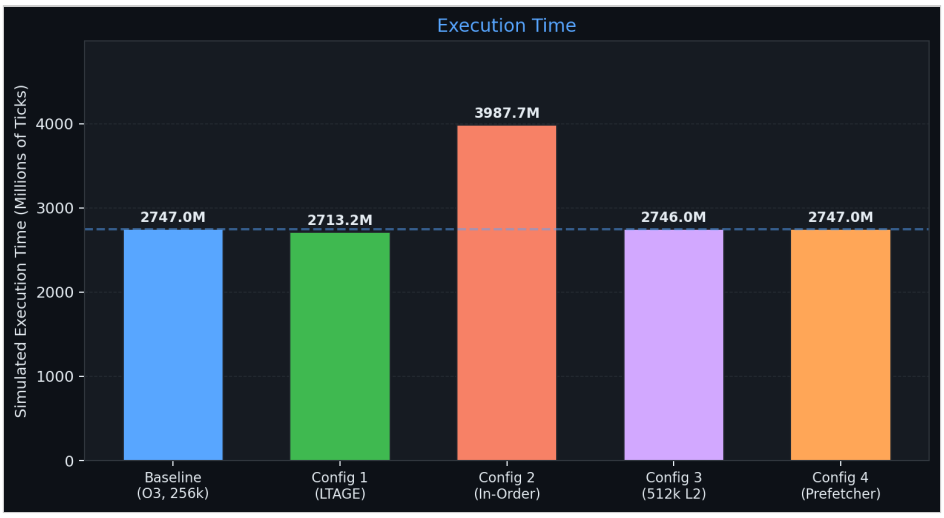

Figure 4: Total simulated execution time (millions of ticks). The graph mirrors the IPC trends, confirming that Config 2 suffers massive execution time penalties due to a lack of latency hiding capabilities.

6. Architectural Bottleneck Analysis

A. The Control Flow Bottleneck (Branch Prediction)

The data-dependent swaps during Quicksort's array partitioning act as a control flow hurdle. The Baseline `TournamentBP` suffered a Branch MPKI of ~1.00. Upgrading to the **LTAGE Predictor (Config 1)** dropped the Branch MPKI to 0.10. LTAGE successfully leverages long geometric histories to decipher complex, data-dependent loops. However, despite a substantial drop in Branch MPKI, overall IPC only increased from 0.861 to 0.872. This firmly establishes that while branch prediction improves execution efficiency, it is **not the dominant bottleneck**.

B. The Memory Latency Bottleneck (Compute Pipeline)

To measure exactly how heavily the architecture relies on masking memory latency, we removed the Out-of-Order capabilities (Config 2). Without OoO scheduling, IPC plummeted to 0.593 and execution time spiked by roughly 45%. **Out-of-order execution significantly improves performance by enabling latency hiding through instruction-level parallelism (ILP) and memory-level parallelism (MLP).** The O3 CPU allows the processor to overlap cache misses and maintain pipeline throughput while waiting on slow DRAM responses.

C. The L2 Cache Capacity Sensitivity

Doubling the L2 cache size from 256kB to 512kB (Config 3) dropped the L2 MPKI by an imperceptible margin, yielding effectively identical IPC. **Within the evaluated cache size range (256kB–512kB), performance shows minimal sensitivity to capacity, suggesting limited benefit from increasing L2 size for this workload.** The recursive bounds swapping reduces temporal locality across recursive subproblems and introduces irregular access patterns that cache capacity scaling cannot resolve.

D. The Efficacy of Spatial Prefetching

We attached a standard `StridePrefetcher` to the L2 Cache (Config 4) to exploit the sequential scans during the `partition()` loop. The IPC and L2 MPKI remained identical to the Baseline. We note that an identical IPC may indicate prefetcher ineffectiveness or

underlying configuration limitations within the simulation. **We hypothesize that stride prefetchers may fail here due to short sequential regions and irregular recursion boundaries.** The sequential scans within `partition()` may be too short-lived to effectively train the prefetcher. Furthermore, the L1D cache naturally filters sequential accesses, meaning the L2 cache observes primarily the unpredictable, erratic bounds updates with limited spatial locality.

7. Qualitative PPA Analysis & Conclusion

Because synthesis tools (e.g., McPAT or CACTI) were not utilized, this is a qualitative assessment of Power, Performance, and Area (PPA) tradeoffs:

- **The 512kB L2 Upgrade:** Doubling LLC capacity likely increases area and power due to larger SRAM structures. Given the near-zero performance gain, this is a poor architectural tradeoff for this specific workload.
- **The StridePrefetcher:** Consumes area for tracking tables but provided zero measurable performance utility here, meaning it provides poor efficiency in this scenario.
- **The LTAGE Predictor:** Demands an increase in SRAM storage for its history tables. However, by significantly reducing the Branch MPKI, it mitigates the dynamic power waste associated with flushing the pipeline and executing wrong-path instructions.

> **Conclusion:** The optimal architecture for executing Recursive Quicksort among the tested configurations is the **O3CPU + LTAGE Predictor + 256kB L2 Cache**. Out-of-order execution is highly effective at hiding the unavoidable L2 miss penalties through ILP and MLP. The LTAGE predictor stabilizes control flow, while the 256kB L2 avoids the diminishing returns of capacity over-scaling. Attempting to solve memory delays through brute-force cache scaling or simple spatial prefetching is ineffective for algorithms characterized by such mixed locality.